

ФГБОУ ВО «Пензенский государственный университет»
Политехнический институт
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №3
По дисциплине «Логика и основы
алгоритмизации в инженерных задачах»
На тему «Динамические списки»

Выполнили: ст. гр. 25BBB1
Разживалов Н.А.
Цеплов Е.В.
Проверили: к.т.н., доцент Юрова О.В.
к.т.н., доцент Деев М.В.

Пенза 2026

Название

Динамические списки.

Цель работы

Научиться реализовывать динамические структуры данных:

приоритетную очередь, очередь, стек.

Лабораторное задание

1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
2. На основе приведенного кода реализуйте структуру данных *Очередь*.
3. На основе приведенного кода реализуйте структуру данных *Стек*.

Ход работы

В образце структуры добавили поле приоритета, где будет храниться приоритет элемента, влияющий на порядок в очереди.

```
struct node
{
    char inf[256];
    int priority;
    struct node* next;
};

struct node* head = NULL;
```

Тело main реализовали бесконечным циклом while с конструкцией switch для дальнейшего выбора действия

```
int main()
{
    setlocale(LC_ALL, "Russian");
    int choice;

    while (1){
        printf("Выберете действие и введите его номер.\n 1 - добавить элемент\n");
        scanf_s("%d", &choice);

        switch (choice)
        {
            case 1:
                add();
                break;

            case 2:
                review();
                break;

            case 3:
                find();
                break;

            case 4:
                del();
                break;

            case 5:
                clear();
                break;

            case 6:
                return 0;

            default:
                printf("Некорректный символ\n");
        }
    }
}
```

```

struct node* get_struct()
{
    struct node* p;

    p = (struct node*)malloc(sizeof(struct node));

    if (p == NULL)
    {
        printf("Ошибка памяти\n");
    }

    printf("Введите название ");
    scanf_s("%s", p->inf, (unsigned)sizeof(p->inf));

    printf("Введите приоритет ");
    scanf_s("%d", &p->priority);

    p->next = NULL;

    return p;
}

void add()
{
    struct node* p;
    struct node* q;

    p = get_struct();

    if (head == NULL)
    {
        head = p;
        return;
    }

    if (p->priority > head->priority)
    {
        p->next = head;
        head = p;
        return;
    }

    q = head;

    while (q->next != NULL &&
           q->next->priority >= p->priority)
    {
        q = q->next;
    }

    p->next = q->next;
    q->next = p;
}

```

Внесли изменения в get_struct и функцию add. Адрес нового элемента помещаем в указатель p, через который обращаемся к элементам структуры. Через указатель q проходим последовательно по элементам структуры до первого элемента с меньшим приоритетом, чтобы поместить элемент перед ним. Если первый элемент пуст, то сразу перемещаем в начало

В функции review и find включили в вывод приоритет.

```

void review()
{
    struct node* p = head;

    if (head == NULL)
    {
        printf("Список пуст\n");
        return;
    }

    printf("Список\n");

    while (p != NULL)
    {
        printf("Название: %s, приоритет: %d\n", p->inf, p->priority);

        p = p->next;
    }
}

```

```

void find()
{
    char name[256];
    struct node* p = head;

    printf("Введите элемент, который нужно найти ");
    scanf_s("%s", name, (unsigned)sizeof(name));

    while (p != NULL)
    {
        if (strcmp(name, p->inf) == 0)
        {
            printf("Элемент \"%s \", приоритет = %d\n", p->inf, p->priority);
            return;
        }

        p = p->next;
    }

    printf("Элемент не найден\n");
}

```

Функцию del оставили почти без изменений

```

void del()
{
    char name[256];
    struct node* p = head;
    struct node* prev = NULL;

    printf("Введите название элемента, который нужно удалить ");
    scanf_s("%s", name, (unsigned)sizeof(name));

    if (head != NULL && strcmp(name, head->inf) == 0)
    {
        head = head->next;
        free(p);

        printf("Элемент удалён\n");
        return;
    }

    while (p != NULL)
    {
        if (strcmp(name, p->inf) == 0)
        {
            prev->next = p->next;
            free(p);

            printf("Элемент удалён\n");
            return;
        }

        prev = p;
        p = p->next;
    }

    printf("Элемент не найден\n");
}

```

Приступили к реализации очереди. Убрали приоритет и добавили помимо указателя на начало указатель на конец.

```

struct node
{
    char inf[256];
    struct node* next;
};

struct node* head = NULL;
struct node* last = NULL;

```

В функции добавления элемента убрали приоритет и реализовали добавление элемента всегда в конец.

```
void add()
{
    struct node* p;

    p = (struct node*)malloc(sizeof(struct node));

    if (p == NULL)
    {
        printf("Ошибка памяти\n");
        return;
    }

    printf("Введите название ");
    scanf_s("%s", p->inf, (unsigned)sizeof(p->inf));

    p->next = NULL;

    if (head == NULL)
    {
        head = p;
        last = p;
    }
    else
    {
        last->next = p;
        last = p;
    }
}
```

В функции удаления элемента убрали выбор удаляемого элемента и заменили на удаление всегда первого элемента.

```
void del()
{
    struct node* p;

    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    p = head;
    head = head->next;

    free(p);

    if (head == NULL)
        last = NULL;
}
```

В остальных функциях убрали приоритет, серьёзных изменений в принцип их работы вносить не потребовалось.

```
void find()
{
    char name[50];
    struct node* p = head;
    int i = 1;

    printf("Введите название элемента ");
    scanf_s("%s", name, (unsigned)sizeof(name));

    while (p != NULL)
    {
        if (strcmp(name, p->inf) == 0)
        {
            printf("Элемент есть в очереди %s под номером %d\n", p->inf, i);
            return;
        }
        i++;
        p = p->next;
    }

    printf("Элемент не найден\n");
}

void review()
{
    struct node* p = head;
    int i = 1;

    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    while (p != NULL)
    {
        printf("%d. %s\n", i, p->inf);
        p = p->next;
        i++;
    }
}
```

Приступили к реализации стека. Убрали указатель на последний элемент, поскольку для стека он не требуется.

```
struct node
{
    char inf[256];
    struct node* next;
};

struct node* head = NULL;
```

Заменили названия add на push и del на pop, по аналогии с ассемблером. Изменили добавление в конец на добавление в начало.

```
void push()
{
    struct node* p;

    p = (struct node*)malloc(sizeof(struct node));

    if (p == NULL)
    {
        printf("Ошибка памяти\n");
        return;
    }

    printf("Введите название ");
    scanf_s("%s", p->inf, (unsigned)sizeof(p->inf));

    p->next = head;
    head = p;
}
```

Поскольку последний элемент уже находится в начале удаляем его.

```
void pop()
{
    struct node* p;

    if (head == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    p = head;
    head = head->next;

    free(p);
}
```

В остальных функциях только меняем очередь на стек в выводе, и не вносим серьёзных изменений.

```
void review()
{
    struct node* p = head;
    int i = 1;

    if (head == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    while (p != NULL)
    {
        printf("%d. %s\n", i, p->inf);
        p = p->next;
        i++;
    }
}

void find()
{
    char name[50];
    struct node* p = head;
    int i = 1;

    printf("Введите название элемента");
    scanf_s("%s", name, (unsigned)sizeof(name));

    while (p != NULL)
    {
        if (strcmp(name, p->inf) == 0)
        {
            printf("Элемент есть в стеке %s под номером %d\n", p->inf, i);
            return;
        }
        i++;
        p = p->next;
    }

    printf("Элемент не найден\n");
}
```

Псевдокод

Приоритетная очередь:

ФУНКЦИЯ get_struct

СОЗДАТЬ новый элемент p

ЕСЛИ память не выделена

ТО

ВЫВЕСТИ "Ошибка памяти"

ЗАВЕРШИТЬ

КОНЕЦ ЕСЛИ

ВВЕСТИ p.inf

ВВЕСТИ p.priority

p.next ← NULL

ВОЗВРАТ p

КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ add

p ← get_struct()

ЕСЛИ head = NULL

ТО

head ← p

ВОЗВРАТ

КОНЕЦ ЕСЛИ

ЕСЛИ p.priority > head.priority

ТО

p.next ← head

head ← p

ВОЗВРАТ

КОНЕЦ ЕСЛИ

```
q ← head
ПОКА q.next ≠ NULL
    И q.next.priority ≥ p.priority ДЕЛАТЬ
    q ← q.next
КОНЕЦ ПОКА
p.next ← q.next
q.next ← p
КОНЕЦ ФУНКЦИИ
```

Очередь:

```
ФУНКЦИЯ add
    СОЗДАТЬ новый элемент p
    ВВЕСТИ p.inf
    p.next ← NULL
    ЕСЛИ head = NULL
        ТО
            head ← p
            last ← p
        ИНАЧЕ
            last.next ← p
            last ← p
    КОНЕЦ ЕСЛИ
КОНЕЦ ФУНКЦИИ
```

```
ФУНКЦИЯ del
    ЕСЛИ head = NULL
        ТО
            ВЫВЕСТИ "Очередь пуста"
        ВОЗВРАТ
```

```
    КОНЕЦ ЕСЛИ
    p ← head
    head ← head.next
    УДАЛИТЬ p
    ЕСЛИ head = NULL
        ТО
            last ← NULL
    КОНЕЦ ЕСЛИ
КОНЕЦ ФУНКЦИИ
```

Стек:

```
ФУНКЦИЯ push
    СОЗДАТЬ новый элемент p
    ВВЕСТИ p.inf
    p.next ← head
    head ← p
КОНЕЦ ФУНКЦИИ
```

```
ФУНКЦИЯ pop
    ЕСЛИ head = NULL
        ТО
            ВЫВЕСТИ "Стек пуст"
            ВОЗВРАТ
    КОНЕЦ ЕСЛИ
    p ← head
    head ← head.next
    УДАЛИТЬ p
КОНЕЦ ФУНКЦИИ
```

Одинаковые для всех заданий:

ФУНКЦИЯ review

$p \leftarrow \text{head}$

$i \leftarrow 1$

ЕСЛИ $\text{head} = \text{NULL}$

ТО

ВЫВЕСТИ "Список пуст"

ВОЗВРАТ

КОНЕЦ ЕСЛИ

ПОКА $p \neq \text{NULL}$ ДЕЛАТЬ

ВЫВЕСТИ $i, p.\text{inf}$

$i \leftarrow i + 1$

$p \leftarrow p.\text{next}$

КОНЕЦ ПОКА

КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ find

ВВЕСТИ name

$p \leftarrow \text{head}$

$i \leftarrow 1$

ПОКА $p \neq \text{NULL}$ ДЕЛАТЬ

ЕСЛИ $p.\text{inf} = \text{name}$

ТО

ВЫВЕСТИ "Элемент найден"

ВЫВЕСТИ "Номер:", i

ВЫВЕСТИ "Название:", $p.\text{inf}$

ВОЗВРАТ

КОНЕЦ ЕСЛИ

$i \leftarrow i + 1$

$p \leftarrow p.next$

КОНЕЦ ПОКА

ВЫВЕСТИ "Элемент не найден"

КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ clear

ПОКА $head \neq NULL$ ДЕЛАТЬ

$p \leftarrow head$

$head \leftarrow head.next$

УДАЛИТЬ p

КОНЕЦ ПОКА

КОНЕЦ ФУНКЦИИ

Результаты работы программы

Приоритетная очередь:

```
Выберете действие и введите его номер.
1 - добавить элемент в список
2 - вывести список
3 - найти элемент в списке
4 - удалить элемент из списка
5 - очистить список
6 - выход
1
Введите название A
Введите приоритет 3
Выберете действие и введите его номер.
1 - добавить элемент в список
2 - вывести список
3 - найти элемент в списке
4 - удалить элемент из списка
5 - очистить список
6 - выход
1
Введите название B
Введите приоритет 5
Выберете действие и введите его номер.
1 - добавить элемент в список
2 - вывести список
3 - найти элемент в списке
4 - удалить элемент из списка
5 - очистить список
6 - выход
1
Введите название C
Введите приоритет 1
Выберете действие и введите его номер.
1 - добавить элемент в список
2 - вывести список
3 - найти элемент в списке
4 - удалить элемент из списка
5 - очистить список
6 - выход
2
Список
Название: B, приоритет: 5
Название: A, приоритет: 3
Название: C, приоритет: 1
Выберете действие и введите его номер.
1 - добавить элемент в список
2 - вывести список
3 - найти элемент в списке
4 - удалить элемент из списка
5 - очистить список
6 - выход
3
Введите элемент, который нужно найти B
Элемент " B ", приоритет = 5
```

```
Введите название элемента, который нужно удалить В
Элемент удалён
Выберете действие и введите его номер.
  1 – добавить элемент в список
  2 – вывести список
  3 – найти элемент в списке
  4 – удалить элемент из списка
  5 – очистить список
  6 – выход
2
Список
Название: А, приоритет: 3
Название: С, приоритет: 1
Выберете действие и введите его номер.
  1 – добавить элемент в список
  2 – вывести список
  3 – найти элемент в списке
  4 – удалить элемент из списка
  5 – очистить список
  6 – выход
5
Выберете действие и введите его номер.
  1 – добавить элемент в список
  2 – вывести список
  3 – найти элемент в списке
  4 – удалить элемент из списка
  5 – очистить список
  6 – выход
2
Список пуст
```

Очередь:

```
Выберете действие и введите его номер.
  1 – добавить элемент в очередь
  2 – вывести очередь
  3 – найти элемент в очереди
  4 – удалить элемент из очереди
  5 – очистить очередь
  6 – выход
1
Введите название А
Выберете действие и введите его номер.
  1 – добавить элемент в очередь
  2 – вывести очередь
  3 – найти элемент в очереди
  4 – удалить элемент из очереди
  5 – очистить очередь
  6 – выход
1
Введите название В
Выберете действие и введите его номер.
  1 – добавить элемент в очередь
  2 – вывести очередь
  3 – найти элемент в очереди
  4 – удалить элемент из очереди
  5 – очистить очередь
  6 – выход
1
Введите название С
```

Выберете действие и введите его номер.

- 1 - добавить элемент в очередь
- 2 - вывести очередь
- 3 - найти элемент в очереди
- 4 - удалить элемент из очереди
- 5 - очистить очередь
- 6 - выход

2

- 1. А
- 2. В
- 3. С

Выберете действие и введите его номер.

- 1 - добавить элемент в очередь
- 2 - вывести очередь
- 3 - найти элемент в очереди
- 4 - удалить элемент из очереди
- 5 - очистить очередь
- 6 - выход

3

Введите название элемента В

Элемент есть в очереди В под номером 2

Выберете действие и введите его номер.

- 1 - добавить элемент в очередь
- 2 - вывести очередь
- 3 - найти элемент в очереди
- 4 - удалить элемент из очереди
- 5 - очистить очередь
- 6 - выход

4

Выберете действие и введите его номер.

- 1 - добавить элемент в очередь
- 2 - вывести очередь
- 3 - найти элемент в очереди
- 4 - удалить элемент из очереди
- 5 - очистить очередь
- 6 - выход

2

- 1. В
- 2. С

Стек:

Выберете действие и введите его номер.

- 1 - добавить элемент в стек
- 2 - вывести стек
- 3 - найти элемент в стеке
- 4 - удалить элемент из стека
- 5 - очистить стек
- 6 - выход

2

- 1. С
- 2. В
- 3. А

Выберете действие и введите его номер.

- 1 - добавить элемент в стек
- 2 - вывести стек
- 3 - найти элемент в стеке
- 4 - удалить элемент из стека
- 5 - очистить стек
- 6 - выход

3

Введите название элементаВ

Элемент есть в стеке В под номером 2

```
4
Выберете действие и введите его номер.
1 - добавить элемент в стек
2 - вывести стек
3 - найти элемент в стеке
4 - удалить элемент из стека
5 - очистить стек
6 - выход
2
1. В
2. А
```

Вывод

Научились реализовывать динамические структуры данных:
приоритетную очередь, очередь, стек.